Research on Feature Extraction of Tumor Image Based on

Convolutional Neural Network

Abstract

Tumor detection and classification are essential tasks in medical imaging for early diagnosis and effective treatment of cancer. In recent years, image processing techniques have been used to develop automated and accurate systems for tumor detection and classification. This paper presents a review of different methods used in tumor detection and classification, including image segmentation, feature extraction, and classification algorithms. The focus is on the application of image processing techniques to different medical imaging modalities such as Magnetic Resonance Imaging (MRI), Computed Tomography (CT), and Ultrasound. The paper also discusses the challenges and limitations associated with tumor detection and classification using image processing, as well as the potential future directions for research in this field. The results show that image processing techniques have great potential for improving the accuracy and efficiency of tumor detection and classification in medical imaging.

Keywords: *Magnetic Resonance Imaging (MRI), Image Processing*

Table of Contents

Abstract 2

Chapter 1 5

Introduction 5

1.1 Research Question: 5

1.2 Aim and Objective 6

Chapter 2 7

Related Work and Background 7

2.1 Literature Review 7

2.3 Technical Background 9

2.3.1 Artificial Intelligence 9

2.3.2 Deep Learning or Deep Structured Learning (DL) 10

Chapter 3 12

Methodologies 12

3.1 Convolution Neural Network (CNN) 12

3.2 Evaluation Metrics 13

3.2.1 Graphs 13

3.2.2 Report on Classification 14

3.2.3 Confusion Matrix: 14

3.3 Python 16

3.3.1 Anaconda Framework 17

3.3.2Jupyter Notebook 17

3.4 NumPy 18

3.5 Matoplotlib 19

Chapter 4 20

4.1 System Design & Implementation 20

Chapter 5 37

Results Analysis 37

5.1Convolution Neural Network (CNN) 37

5.1.1CLSTM - Report on Classification 37

5.1.2 CNN- Confusion Matrix 38

Chapter 6 39
6.1 Conclusion 39
References 40
Appendix A 41

Chapter 1

Introduction

Magnetic resonance imaging (MRI) provides detailed information about brain tumor anatomy, cellular structure and vascular supply, making it an important tool for the effective diagnosis, treatment and monitoring of the disease. Magnetic resonance imaging (MRI) is a noninvasive medical test that helps physicians diagnose and treat medical conditions. MRI uses a powerful magnetic field, radio frequency pulses and a computer to produce detailed pictures of organs, soft tissues, bone and virtually all other internal body structures. The images can then be examined on a computer monitor, transmitted electronically, printed or copied to a CD. MRI does not use ionizing radiation (x-rays). Detailed MR images allow physicians to evaluate various parts of the body and determine the presence of certain diseases. Automated brain tumor detection from MRI images is one of the most challenging task in today's modern Medical imaging research. Automatic detection requires brain image segmentation, which is the process of partitioning the image into distinct regions, is one of the most important and challenging aspect of computer aided clinical diagnostic tools. Noises present in the Brain MRI images are multiplicative noises and reductions of these noises are difficult task. The minute anatomical details should not be destroyed by the process of noise removal from clinical point of view. These makes accurate segmentation of brain images a challenge. However, accurate segmentation of the MRI images is very important and crucial for the exact diagnosis by computer aided clinical tools. A large variety of algorithms for segmentation of MRI images had been developed. Surgical planning, postsurgical assessment, abnormality detection, and much other medical application require medical image segmentation. In spite of wide number of automatic and semi – automatic image segmentation techniques, they fail in most cases largely because of unknown and irregular noise, in-homogeneity, poor contrast and weak boundaries which are inherent to medical images.

1.1 Research Question:

- RQ1: How can image processing techniques be used to improve the accuracy of tumor detection and classification?
- RQ3: What are the most effective image features and feature extraction methods for tumor detection and classification?

1.2 Aim and Objective

The aim of tumor detection and classification using image processing is to accurately identify and classify different types of tumors in medical images, such as MRI and CT scans. This can help in early diagnosis and treatment of various types of cancers.

The following criteria are listed dependent on the purpose with this survey:

- Developing image processing algorithms that can accurately detect and segment tumors from medical images.
- Developing machine learning models that can classify different types of tumors based on their characteristics.
- Evaluating the performance of the developed algorithms and models using various metrics such as sensitivity, specificity, and accuracy.

Chapter 2

Related Work and Background

2.1 Literature Review

1: DETECTION AND CLASSIFICATION OF BRAIN TUMOR IN MRI IMAGES

Brain tumor is a serious disease occurring in human being. Medical treatment process mainly depends on tumor types and its location. The final decision of neurospecialists and radiologist for the tumor diagnosis mainly depend on evaluation of MRI (Magnetic Resonance Imaging) Images. The manual evaluation process is time-consuming and needs domain expertise to avoid human errors. To overcome this issue, Faster R-CNN deep learning algorithm was proposed for detecting the tumor and marking the area of their occurrence with Region Proposal Network (RPN). The selected MR image dataset consists of three primary brain tumors namely glioma, meningioma and pituitary. The proposed algorithm uses VGG-16 architecture as a base layer for both the region proposal network and the classifier network. Detection and classification results of the algorithm demonstrate that it is able to achieve an average precision of 75.18% for glioma, 89.45% for meningioma and 68.18% for pituitary tumor. As a performance measure, the algorithm achieved a mean average precision of 77.60% for all the classes.

2: AN ADAPTIVE MRI TUMOR DETECTION USING NEURALNETWORK BASED ADABOOST ALGORITHM

The study focused on segmentation effects of the improved algorithm of traditional neural network algorithm, small kernels two-path convolutional neural network (SK-TPCNN) combined with random forest (RF) algorithm on MRI images for patella, and the influencing factors of patellar dislocation during exercise. In this article, the MRI images for patellar dislocation patients were detected by virtue of the neural network algorithm, to establish the patella-related MRI image segmentation algorithm. In terms of dice similarity coefficient (DSC), positive predictive value (PPV), and sensitivity, the detection accuracy of MRI images for patella was

evaluated, and the segmentation effect of MRI images for patella was assessed. 30 patients, who were diagnosed as patellar dislocation patients in hospital, were chosen as the research subjects. No matter whether the MRI images of the patients went through the processing of the neural network algorithm or not, all of them were analyzed. The results showed that, among the traditional neural network algorithm, SK-TPCNN algorithm, and SK-TPCNN + RF algorithm, the DSC values were 0.82, 0.71, and 0.79, respectively; the PPV values were 0.77, 0.59, and 0.85, respectively; and the sensitivity values were 0.79, 0.62, and 0.89, respectively. Obviously, the various parameters of the SK-TPCNN + RF algorithm were significantly greater than those of the SK-TPCNN algorithm, and the difference was statistically significant (). It indicated that the segmentation ability of MRI images for patella of the NN algorithm was clearly improved, and the MRI image resolution was dramatically raised, which provided a referable basis for the MRI diagnosis of patients with patellar dislocation during exercise.

3: AUTOMATIC BRAIN TUMOR DETECTION IN MAGNETIC RESONANCE IMAGES:

Automatic detection of brain tumor is a difficult task due to variations in type, size, location and shape of tumors. In this paper, a multi-modality framework for automatic tumor detection is presented, fusing different Magnetic Resonance Imaging modalities including T1-weighted, T2-weighted, and T1 with gadolinium contrast agent. The intensity, shape deformation, symmetry, and texture features were extracted from each image. The AdaBoost classifier was used to select the most discriminative features and to segment the tumor region. Multi-modal MR images with simulated tumor have been used as the ground truth for training and validation of the detection method. Preliminary results on simulated and patient MRI show 100% successful tumor detection with average accuracy of 90.11%.

4: A HYBRID APPROACH FOR AUTOMATIC CLASSIFICATION OF BRAIN MRI USING GENETIC ALGORITHM AND SUPPORT VECTOR MACHINE

A hybrid approach for classification of brain tissues in magnetic resonance images (MRI) based on genetic algorithm (GA) and support vector machine (SVM). A wavelet based texture feature set is derived. The optimal texture features are extracted from normal and tumor regions by using spatial gray level dependence method (SGLDM). These features are given as input to the SVM

classifier. The choice of features, which constitute a big problem in classification techniques, is solved by using GA. These optimal features are used to classify the brain tissues into normal, benign or malignant tumor. The performance of the algorithm is evaluated on a series of brain tumor images.

5: BRAIN TUMOR DETECTION BY VARIOUS IMAGE SEGMENTATION TECHNIQUES WITH INTRODUCTION TO NON NEGATIVE MATRIX FACTORIZATION

Image segmentation is one of the fundamental approaches of digital image processing. During past few years, brain tumor segmentation in magnetic resonance imaging (MRI) has become a popular research area in the field of medical imaging system. MRI is used in radiology for analysing internal structures and makes easy to extract the required region. These holding is the simple approach to introduce to the morphological operations which are useful for the detection of the tumor but not all tumor can be specifically detected by this technique so region growing is another technique which provide seed point approach to the segmental ROI region so the tumor is easily detected and also further used for the classification purpose. Nonnegative Matrix Factorization is one of the most promising technique to reduce the dimensionality of the data.NMF has been applied earlier to the image Processing methods such as Pattern analysis and Text mining and now in this paper it is mainly used as a uninterruptable decomposition approach for detection of tumor and to further classify into various types and also for feature extraction.NMF aims to find two non negative matrices whose product closely approximate the original matrix.NMF contains all matrices to contain only non negative elements and the NMF results shows no cancellations, linear super position only and considerable sparsity.

2.3 Technical Background

2.3.1 Artificial Intelligence

Learning by Machine is a measurable study strategy wherein every occurrence of dataset depicts an assortment of elements or properties. But Deep Learning (DL), is a statistical study method that extort properties of actual information. Deep Structured Learning accomplishes this through the use of CNNs of many unseen levels, large amount of information, with dominant computational supply. Each condition appears to be a bit exchangeable; however, with the DL method, the method automatically constructs depiction of information. Compare to, information

depiction in ML algorithms are solid coded with series of attributes, necessitating additional procedures that marks assortment and withdrawal (in Principal Component Analysis).

Such words stand sharply contrasting to Rule-Based Systems, a type of traditional artificial intelligence (AI) systems in which each decision is deliberately built to mimic a statistical model.

In ML and DL there are several distinct models are into categories of two, they are unsupervised and supervised. Unsupervised learning uses algorithms like hierarchical clustering Gauss and mixture and k-Means models to study significant information arrangement. Each instance in the dataset is assigned an output label, where the result is discrete/categorical or real-valued in supervised model. Approximate of real-valued results in Regression technique, although approximate of discrete valued result in classification technique. There are only two output labels in simple binary classification models: 1 (positive) and 0 (negative) (negative).Machine learning includes supervised learning methods such as decision trees, NNs, SVMs, logistic and linear regression, while non-parametric form includes k-closet training.

2.3.2 Deep Learning or Deep Structured Learning (DL)

A machine learning method called deep structured is based on the how people actually learn. Data science, which also includes statistics and predictive modelling, includes it as a critical component. Data scientists who must collect, analyse, and interpret huge amounts of information have profited significantly from DL since it makes the process more difficult and less complicated. and both and unstructured data are used to train deep learning systems. Virtual assistants, autonomous vehicle vision, tax evasion, face recognition as well as many other applications of deep learning are just a few examples.

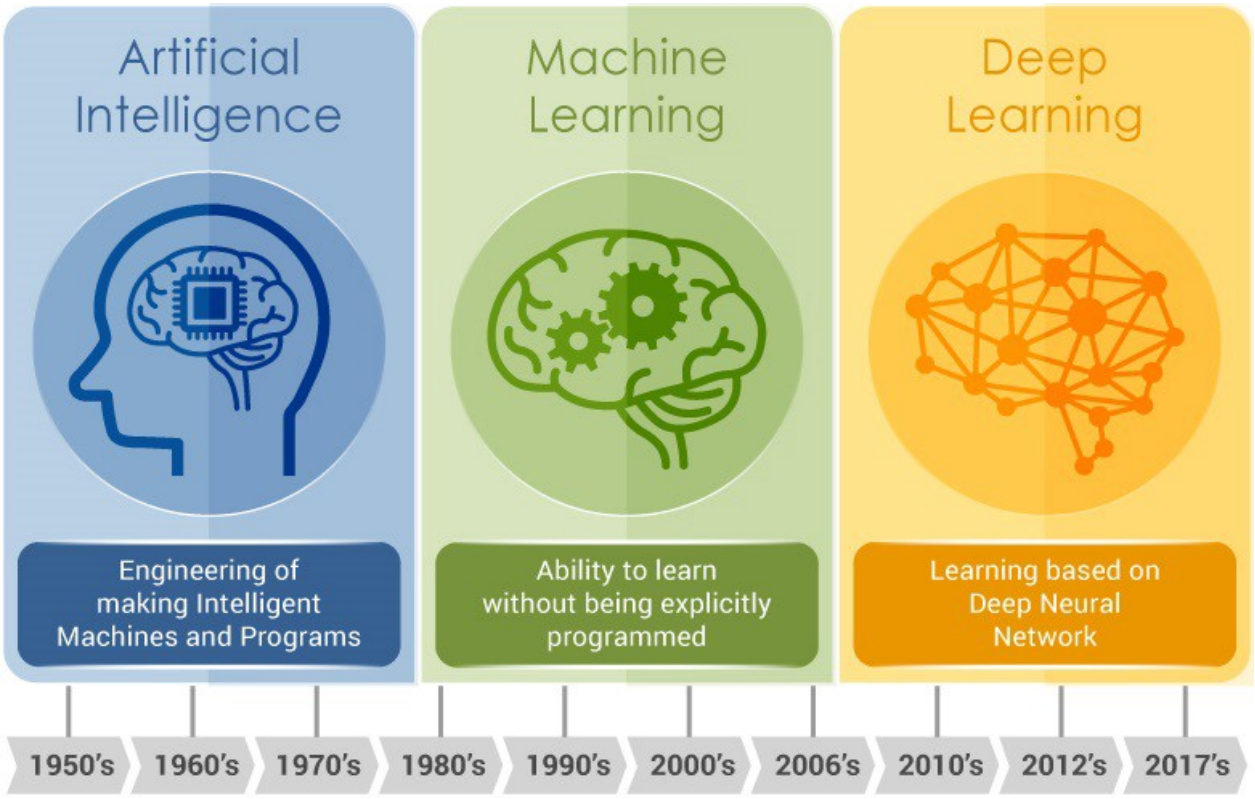


Fig 1: AI, ML, Deep Learning Structure

DL can be regarded of as a means to mechanize predictive analytics at its most fundamental level. Deep learning algorithms, different to typical ML algorithms, are built in a ranking of rising complication and abstraction. DL may be considered as a division of ML. It is a branch of computer science dealing with computer algorithms that learn and evolve all alone. DL uses artificial neural networks (ANN), which are supposed to mimic how individuals think and learn. ML uses simpler methods. Algorithms for DL are employed in a variety of applications, including as medical devices and autonomous driving. DL is being utilized by automobile experts to automatically sense elements like stop signs and traffic signals. In addition, DL is used to recognize pedestrians, which enables to lessen accidents.

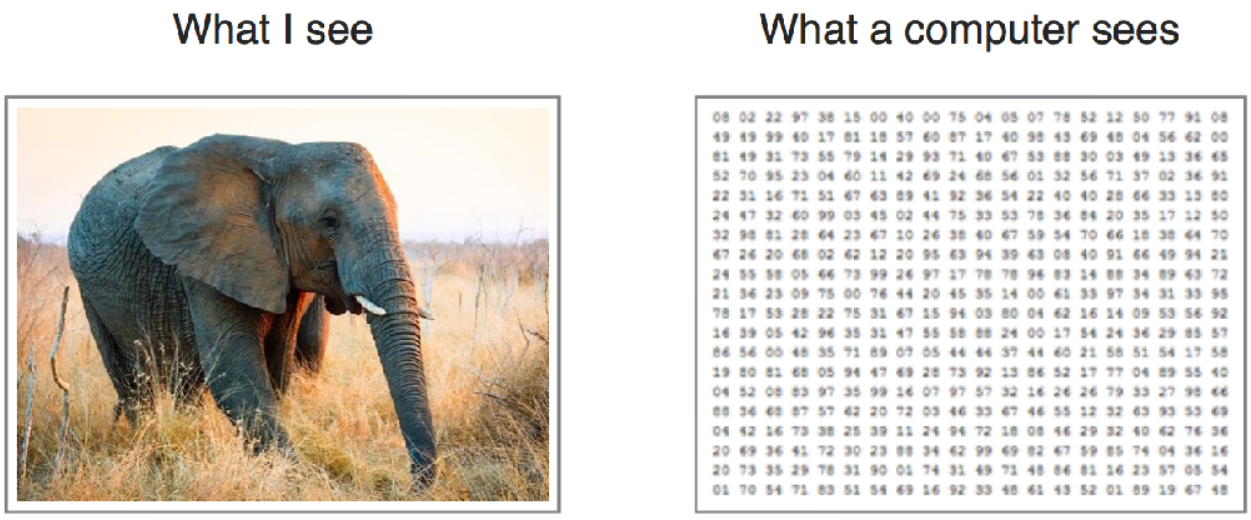
Chapter 3

Methodologies

3.1 Convolution Neural Network (CNN)

A Convolutional neural network (CNN) is a special architecture of artificial neural networks, proposed by Yann LeCun in 1988. CNN uses some features of the visual cortex. One of the most popular uses of this architecture is image classification. For example Facebook uses CNN for automatic tagging algorithms, Amazon—for generating product recommendations and Google for search through among users' photos.

Let us consider the use of CNN for image classification in more detail. The main task of image classification is acceptance of the input image and the following definition of its class. This is a skill that people learn from their birth and are able to easily determine that the image in the picture is an elephant. But the computer sees the pictures quite differently:



Instead of the image, the computer sees an array of pixels. For example, if image size is 300 x 300. In this case, the size of the array will be 300x300x3. Where 300 is width, next 300 is height and 3 is RGB channel values. The computer is assigned a value from 0 to 255 to each of these numbers. This value describes the intensity of the pixel at each point.

To solve this problem the computer looks for the characteristics of the base level. In human understanding such characteristics are for example the trunk or large ears. For the computer, these characteristics are boundaries or curvatures. And then through the groups of convolutional layers the computer constructs more abstract concepts.

Parameters

Filters: The number of convolution filters to include in the layer. Default: 32

Width: The size of a single filter. Default: 3

Horizontal stride: the quantity of cells that will be moved while the vector is convolutioned. Standard: 1

Activation: the procedure that will be used on each output component. ReLu by default

Padding: Same results in padding the input such that the output has the same length as the original input. Valid means "no padding".

Use bias: gives a tensor a bias vector addition.

Trainable: Whether or not we want the weight values to vary when we train the training algorithm. When using an auto encoder's encoder portion also as preparation for a further model, for example, one may want to keep certain network components static.

3.2 Evaluation Metrics

3.2.1 Graphs

Accuracy graph:

A basic principle in supervised evaluation is to evaluate on a different data than the training set. This is because the model can over fit, i.e. learn details which happen by chance in the training set instead of real important patterns. Over fitting is detected by observing whether there is a large difference between performance based on performance on the a testing (or validation) set and training data. So when the two curves are close, it means that there is no or little over fitting in accuracy G

Loss graph:

The slope represents how fast the model reaches its best performance. Apart from the question of computational complexity, what matters is to observe that the performance becomes more or less stable at some point. For example, if you had stopped the training at 50 epochs, then there wouldn't be any stabilization of the performance, meaning that the training was stopped too early. On the contrary, you can observe from your charts that performance stabilizes around epoch 90. There's even a little bit of over fitting happening after this, because the model keeps trying to improve but actually starts learning "details" from the training set.

3.2.2 Report on Classification

The classification report displays the classification metrics accuracy, recall, and f1-score on a per-class basis. The measurements were based on true or false negatives, as well as both true and false positives. Positive and negative are merely names for the projected class in this case. There are four methods for figuring out if the forecasts are correct or not.

3.2.3 Confusion Matrix:

A confusion matrix, sometimes known only as an error matrix, is a specific type of table structure that is used to analyze the performance of algorithms. Such a technique is frequently used it for supervised methods (in unsupervised learning it is usually called a matching matrix). The literature contains descriptions of both variants of the matrices, where each row denotes cases in a pure quality and each column denotes instances in a prediction model. Since it is simple to tell whether the system is confusing class labels, the name was picked (i.e. commonly mislabeling one as another).

Here we analyze and measure the effectiveness of our classification model on our prepared image database. We apply precision on screen for proficiency classifier to dissect the exhibition of our fabricated order and contrast it with current strategies. Knowing force of expectation figure of a new instance following figure development becomes an essential matter. At the point when a prescient representation, fabricated utilizing chronicled data, it's normal to consider how it will perform over information that it hasn't seen previously. Some may attempt various form kinds of a similar expectation issue and afterward contrast their forecast execution with figure out which form is awesome to utilize over a certifiable

dynamic situation (e.g., precision). Exactness, review, and other execution markers, generally employed surveying an indicator's exhibition. Frequently utilized exhibition estimates will be spoken about initially, trailed with clarification &examination of specific notable assessing approaches. "Performance Metrics for Predictive Modeling", a coincidence matrix (contingency table or classification matrices) are significant wellspring of execution measurements for order issues & for a 2 category characterization problem displayed on bellow fig8.

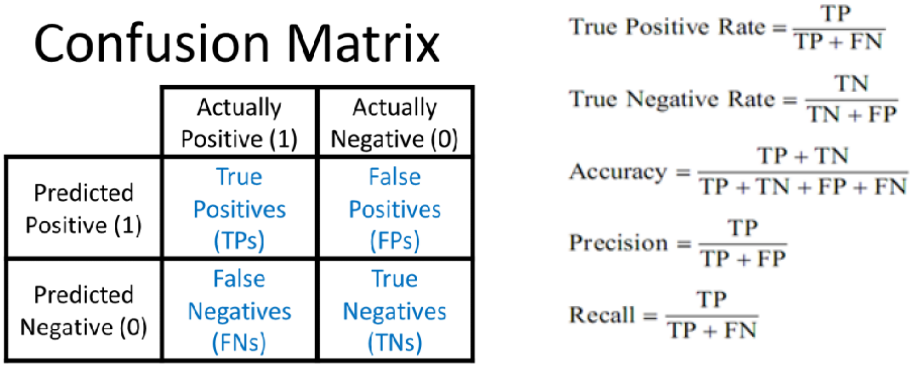


Fig 4: Confusion matrix for CNN and formulas

Glassbox Medicine, "Measuring Performance: The Confusion Matrix," 2019-02-17

In the Fig.12, the qualities that are slantingly from upper-left to bring down right reflect exactchoices, while the qualities outside this corner to corner address blunder. "A classifier's actual positive rate (otherwise called hit rate or review) is determined by partitioning the quantity of accurately arranged up-sides (the genuine positive count) by the absolute number of up-sides. The classifier's by dividing the quantity of incorrectly classified negative (overall fake negative counts) by the overall number of negatives, the bogus positive rate, also known as the bogus alert rate, is calculated. The complete precisely arranged up-sides and negatives separated by the absolute number of tests yields the exactness consequences of a classifier.

Similar to the network of neuron in the human brain, the connection of the vision cortex has an impact on a ConvNet's structure. In the Receptive Field, a small area of the visual field, individual neurons can respond to advancements. To occupy the entire visual field, different comparative fields might be layered on base of one another.

3.3 Python

Preface:

Python employs duck type and has variables with untyped names but typed objects. Type restrictions aren't verified at compile time; instead, actions on an object could fail, indicating that it's not the right type. Despite having a dynamic type system, Python is highly typed, prohibiting actions that are not clearly stated (such as putting an integer to a text) instead of silently attempting to understand them.

Python's class-based system, which is most frequently used for object-oriented programming, enables programmers to define their own kinds. Courses are examples of the metaphysical classifier (already an instance of itself), enabling Meta coding and reflection. New versions of classes are created by invoking the class (for instance, Trash Class () or Eggs Class ()).

Python before version 3.0having two classes: old-style and new-style (both employing the same syntax), [103] only the semantic new style is supported by current Python versions.

Gradual typing is supported in the long run. Static types can be specified using Python's syntax, but the language's C Python implementation does not check them. Compile-time type checking is supported by mypy, an experimental optional static type-checker.

Features:

Python has a wide range of features, a few of which are covered below:

- **Open and Free Source:** By clicking the acquire Python button in the download URL provided below, you may get the Python programming language for freely from the official website. Download since Python is open-source; everyone has access to the source code. As a result, you are free to download, utilize, and share it.
- **Easy to code:** By clicking the acquire Python button in the download URL provided below, you may get the Python programming language for freely from the official website. Download Since Python is open-source, everyone has access to the source code. As a result, you are free to download, utilize, and share it.

- **Simple to Read:** You'll realize that learning Python is really not that difficult. The syntax of Python is rather straightforward, as was already said. The code block is defined by the indentations rather than by semicolons or brackets.
- **Object-Oriented Language:** One of the fundamental features of Python is object-oriented programming. Class, object encapsulation, or other concepts found in object-oriented languages are supported by Python.
- **Support for GUI Programming:** To create graphical user interfaces, Py or a package like PyQt5, PyQt4, wxPython, or PyQt5 could be utilised. The most well-known Python graphic app framework is PyQt5.
- **High-Level Language:** Python is one of these. Python eliminates the need to manage memory or to keep track of the system architecture when writing programmers.
- **Python has an extensible feature:** A language that is extensible. We can translate any Python script into C or C++ and then compile that code using C or C++.

3.3.1 Anaconda Framework

This framework is a Python and R programming language circulation pointed toward improving on bundle the board and arrangement in logical registering (information science, AI applications, enormous scope information handling, prescient examination, etc).
It is renowned as it incorporates a large portion of those apparatuses required for AI and information science in a solitary establishment, making it ideal for speedy and simple sending. Anaconda, like Virtual environment, makes use of the idea of environments to segregate distinct libraries and versions.

3.3.2Jupyter Notebook

The Jupyter is free-source web instrument that lets information researchers make and offer archives with live code, conditions, computational result, representations, and other sight and sound components, just as illustrative message text. R, Scala, Python, Julia and other prominent data science languages are supported by the Jupyter notebook.

To use Jupyter Notebook App:

1. Open a terminal window by clicking on spotlight and typing terminal.
2. In order to enter the starting directory, use cd /some folder name.

3. Enter "Jupyter Notebook" in the search bar to launch the Jupyter Notebook App. box. In a new browser window or tab, the notebook interface will open.

3.4 NumPy

NumPy (numerical Python) is a library that comprises of multi-faceted exhibit objects and a bunch of capacities for overseeing them. NumPy permits you to direct number juggling and coherent procedure on exhibits. This illustration strolls you through the essentials of NumPy, including its design and climate. It likewise covers exhibit capacities, ordering sorts, and different subjects. There's additionally an instructional exercise on the best way to utilize Matoplotlib. For an improved agreement, all of this is introduced utilizing models.

The founder of NumPy, Jim Holguin, created Numeric. Another bundle for some extra components, called Numeracy, was also created.

In 2005, Travis Oliphant built the NumPy bundle by combining the benefits of the Numeracy and Numeric bundles. Numerous donors have supported this open source project.

Tasks based on NumPy

The following tasks can be carried out by an engineer using NumPy:

- Array operations (arithmetical and rational).
- Shape modification using Fourier transformations and algorithms.
- Linear algebra related function. It includes random-number produce &linear algebra functions. ancestor of NumPy

The ndarray type of N-dimensional matrix is considered to be the most important object in NumPy. It alludes to a collection of products that are essentially the same kind. To locate objects in the collection, utilize a zero-based index.

An ndarray's items all take up the same quantity of memory. Data-type object (called **dtype**) is an object in ndarray for each entity.

3.5 Matoplotlib

Plotting library is an excellent Python visualization package for 2dimensional cluster diagrams. They are a different stage information representation bundle in light of NumPy participates and has plans to generally function with the SciPy stack. John Hunter initially presented it in 2002. One of the key advantages of sight is that it gives us visual access to massive amounts of data in just reasonable designs. Matoplotlib has an assortment of plots like disperse, line, histogram, bar, et.

Chapter 4

4.1 System Design & Implementation

System study is the first stage of system development life cycle. This is a brief investigation of the system under consideration and gives a clear picture of what actually the physical system is? In practice, the initial system study involves the preparation of a System proposal which lists the Problem Definition, Objectives of the Study, and Terms of reference for Study, Constraints, Expected benefits of the new system, etc. in the light of the user requirements. The system proposal is prepared by the System Analyst (who studies the system) and places it before the user management. The management may accept the proposal and the cycle proceeds to the next stage. The management may also reject the proposal or request some modifications in the proposal. In summary, we would say that system study phase passes through the following steps:

- problem identification and project initiation
- background analysis
- inference or findings

SYSTEM ARCHITECTURE

System Architecture design-identifies the overall hypermedia structure for the WebApp. Architecture design is tied to the goals establish for a WebApp, the content to be presented, the users who will visit, and the navigation philosophy that has been established. Content architecture, focuses on the manner in which content objects and structured for presentation and navigation. WebApp architecture, addresses the manner in which the application is structure to manage user interaction, handle internal processing tasks, effect navigation, and present content. WebApp architecture is defined within the context of the development environment in which the application is to be implemented.

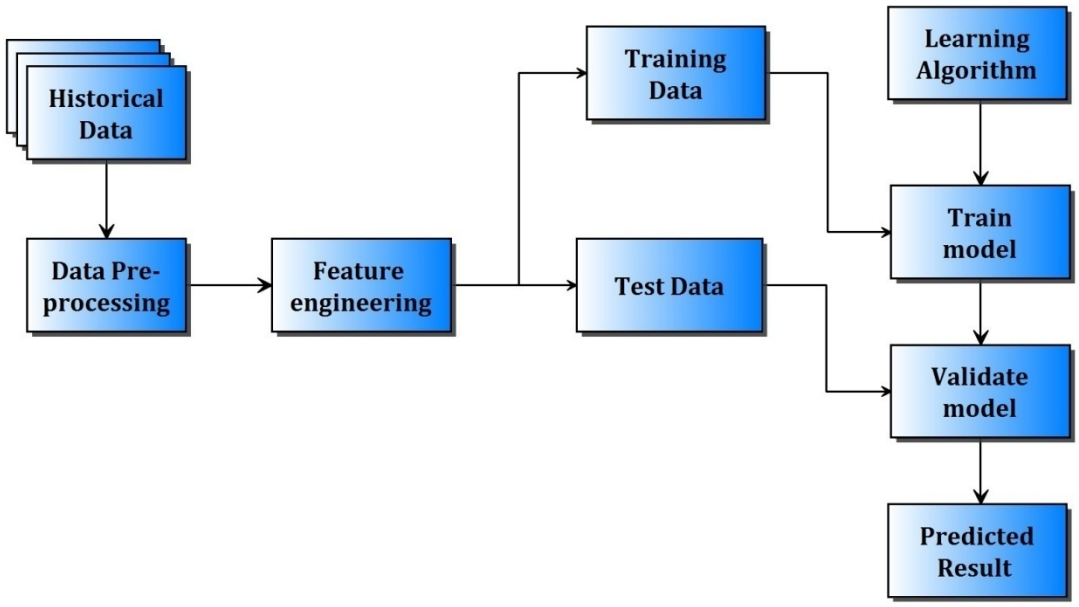


Fig 5: System Architecture

Data flow diagram

A data flow diagram (DFD) is graphic representation of the "flow" of data through an information system. A data flow diagram can also be used for the visualization of data processing (structured design). It is common practice for a designer to draw a context level DFD first which shows the interaction between the system and outside entities. DFD's show the flow of data from external entities into the system, how the data moves from one process to another, as well as its logical storage. There are only four symbols:

Squares representing external entities, which are sources and destinations of information entering and leaving the system. Rounded rectangles representing processes, in other methodologies, may be called 'Activities', 'Actions', 'Procedures', 'Subsystems' etc. which take data as input, do processing to it, and output it. Arrows representing the data flows, which can either, be electronic data or physical items. It is impossible for data to flow from data store to data store except via a process, and external entities are not allowed to access data stores directly. The flat three-sided rectangle is representing data stores should both receive information for storing and provide it for further processing.

Data Flow Diagram level 0

A Data Flow Diagram (DFD) Level 0 is a graphical representation of a system that shows the system's input, processing, and output. In the case of traffic sign recognition, a DFD Level 0 diagram would show the flow of data from the input, which would be a video feed from a camera, to the processing stage, where the image would be analyzed and the traffic signs recognized, and finally to the output, which could be a display of the recognized signs. At Level 0, the DFD would be a high-level overview of the system, showing only the major processes and data flows. The DFD would not include details about the algorithms or techniques used for traffic sign recognition, but rather it would focus on the inputs and outputs of the system. A DFD Level 0 for traffic sign recognition would provide a clear visual representation of the flow of data through the system, helping to identify potential bottlenecks or areas for optimization in the system design.

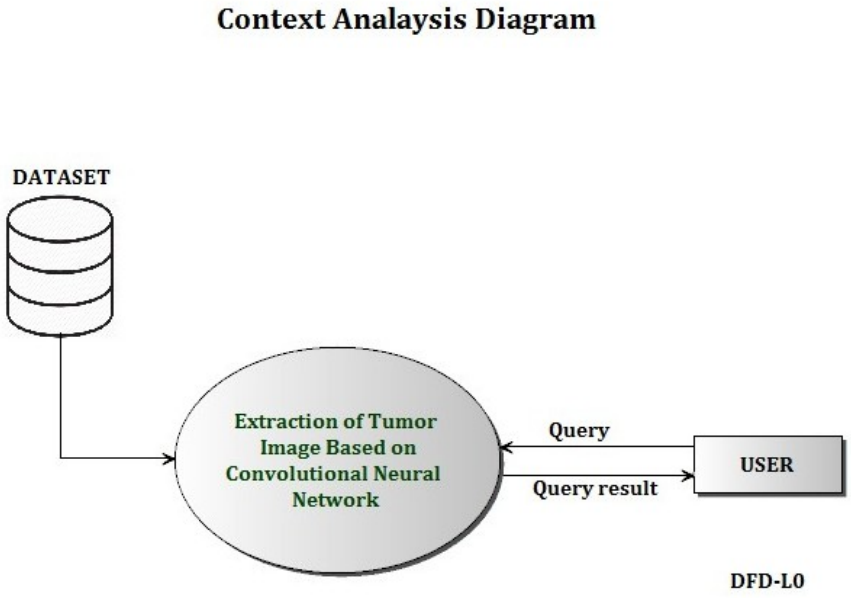


Fig 6: Data Flow Diagram level 0

Data Flow Diagram level 1

A Data Flow Diagram (DFD) Level 1 is a more detailed representation of a system than a Level 0 diagram. At Level 1, the processes identified in the Level 0 diagram are broken down into sub processes, and the data flows between them are more clearly defined. For traffic sign recognition, a DFD Level 1 diagram would show the detailed processes involved in recognizing traffic signs from the image feed. The diagram might include sub processes such as image pre-processing, feature extraction, classification, and decision-making. The data flows between these processes would be clearly defined, showing how the data is transformed and manipulated as it moves through the system. The Level 1 diagram would provide a more detailed view of the system than the Level 0 diagram, allowing for a deeper understanding of how the system works and where potential issues may arise. It could be used as a tool for system design and optimization, helping to identify areas where performance could be improved or where additional functionality could be added.

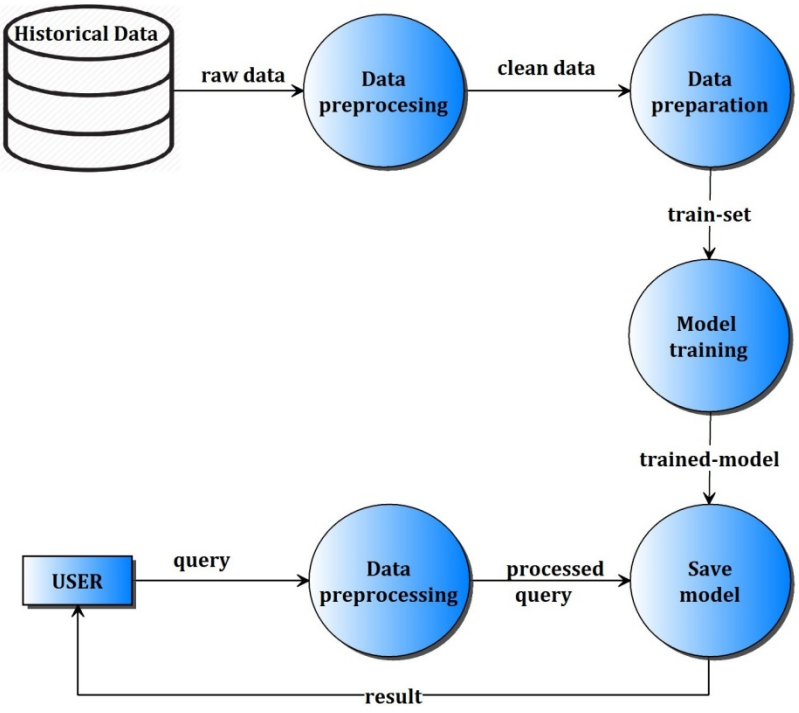


Fig 7: Data Flow Diagram level 1

DFD- Deep Learning Training Prosess

A Level 2 data flow diagram for deep learning training process for traffic sign recognition can be represented as follows:

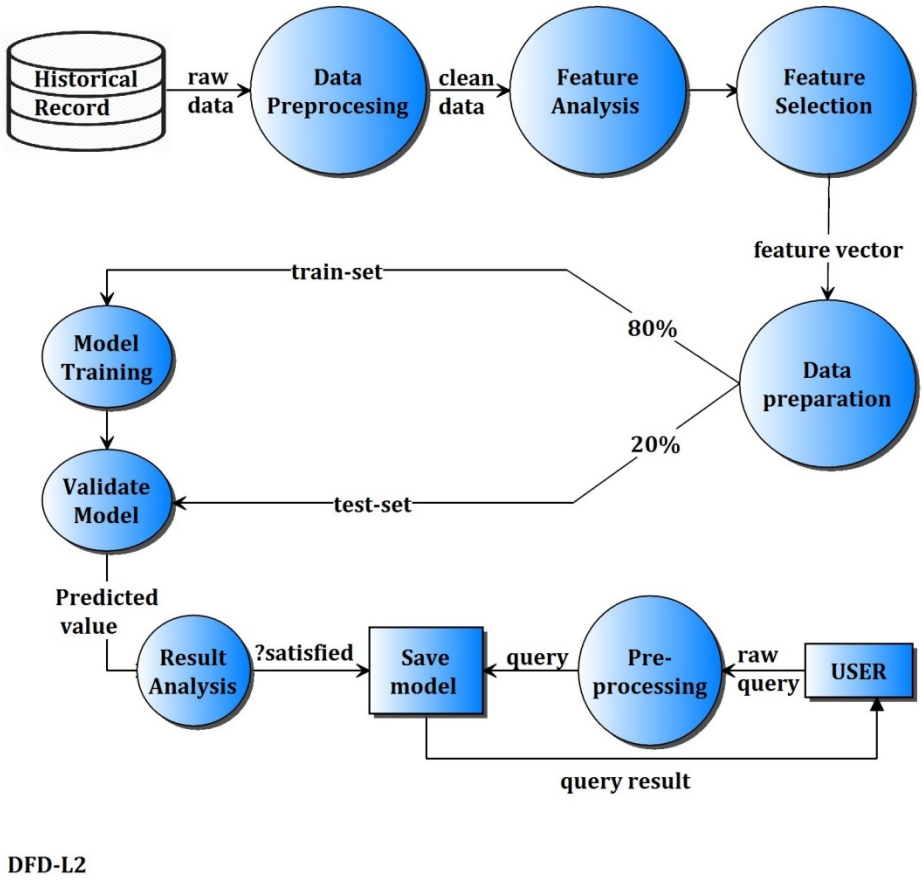


Fig 8: DFD- Deep Learning Training Prosess

Flowchart

It is important to complete all tasks and meet deadlines. There are many project management tools that are available to help project managers manage their tasks and schedule and one of them is the flowchart.

A flowchart is one of the seven basic quality tools used in project management and it displays the actions that are necessary to meet the goals of a particular task in the most practical sequence. Also called as process maps, this type of tool displays a series of steps with branching possibilities that depict one or more inputs and transforms them to outputs.

The advantage of flowcharts is that they show the activities involved in a project including the decision points, parallel paths, branching loops as well as the overall sequence of processing through mapping the operational details within the horizontal value chain. Moreover, this particular tool is very used in estimating and understanding the cost of quality for a particular process. This is done by using the branching logic of the workflow and estimating the expected monetary returns.

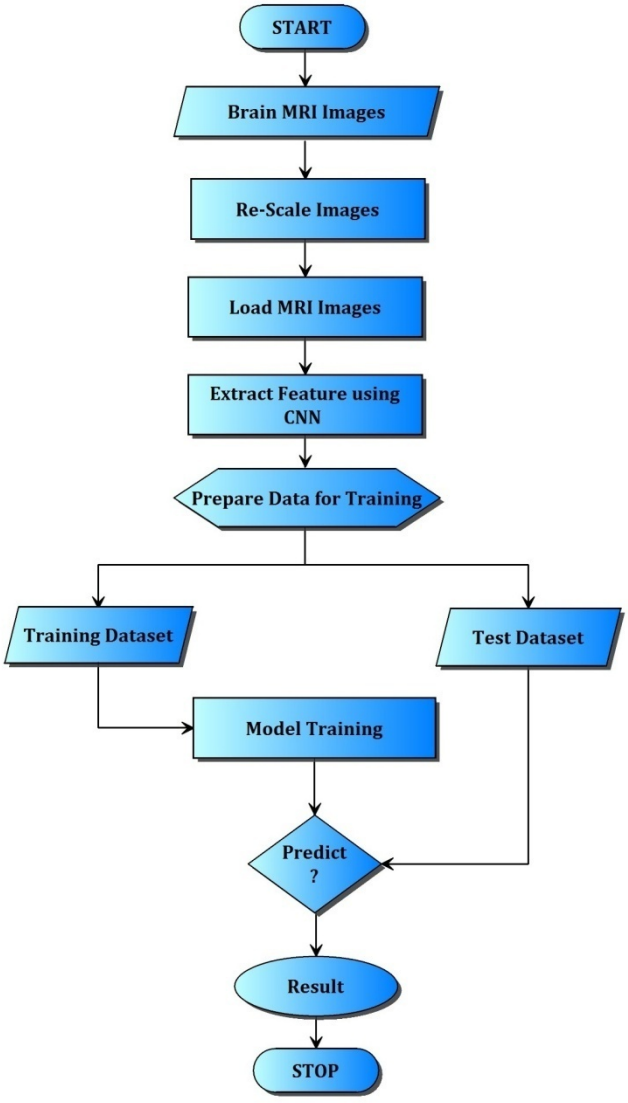


Fig 9: Diagram of Flowchart

Use case Diagram

A use case is a set of scenarios that describing an interaction between a source and a destination. A use case diagram displays the relationship among actors and use cases. The two main components of a use case diagram are use cases and actors. Shows the use case diagram.

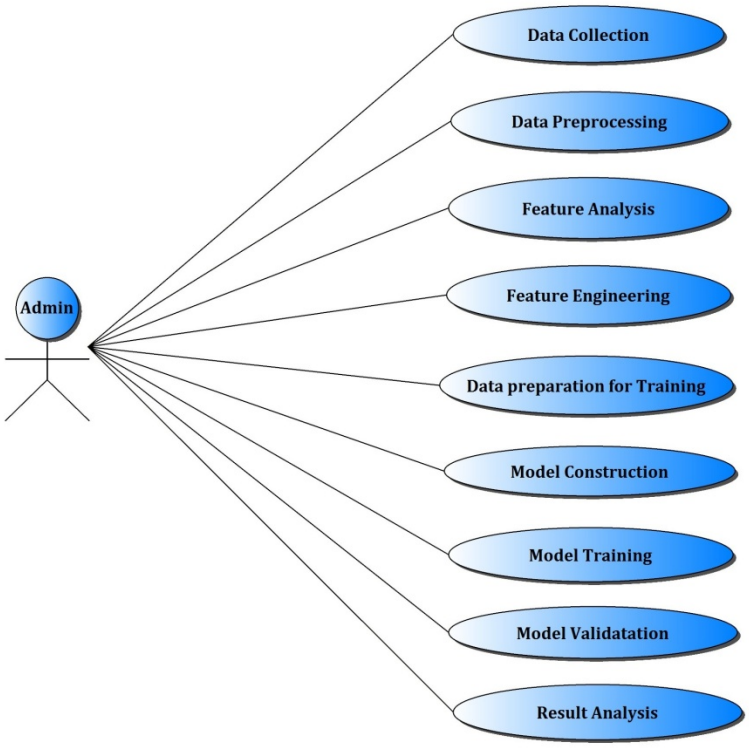


Fig 10: Use case Diagram

Implementations:

SDLC (System Development Life Cycle)

It will cover the details explanation of methodology that is being used to make this project complete and working well. Many methodology or findings from this field mainly generated into journal for others to take advantages and improve as upcoming studies. The method is use to achieve the objective of the project that will accomplish a perfect result. In order to evaluate this project, the methodology based on System Development Life Cycle (SDLC), generally three major step, which is planning, implementing and analysis.

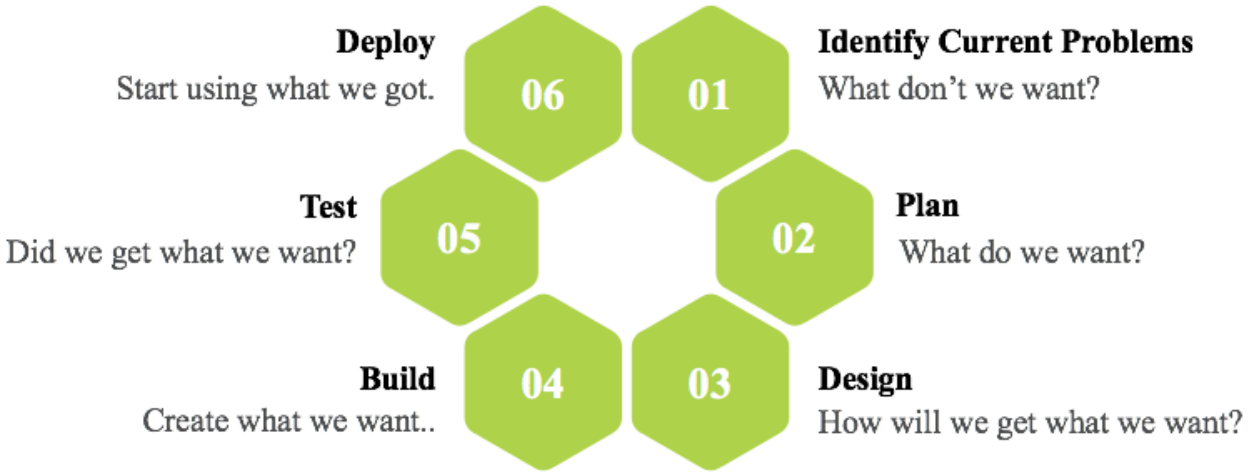


Fig 11: Software Development Life Cycle

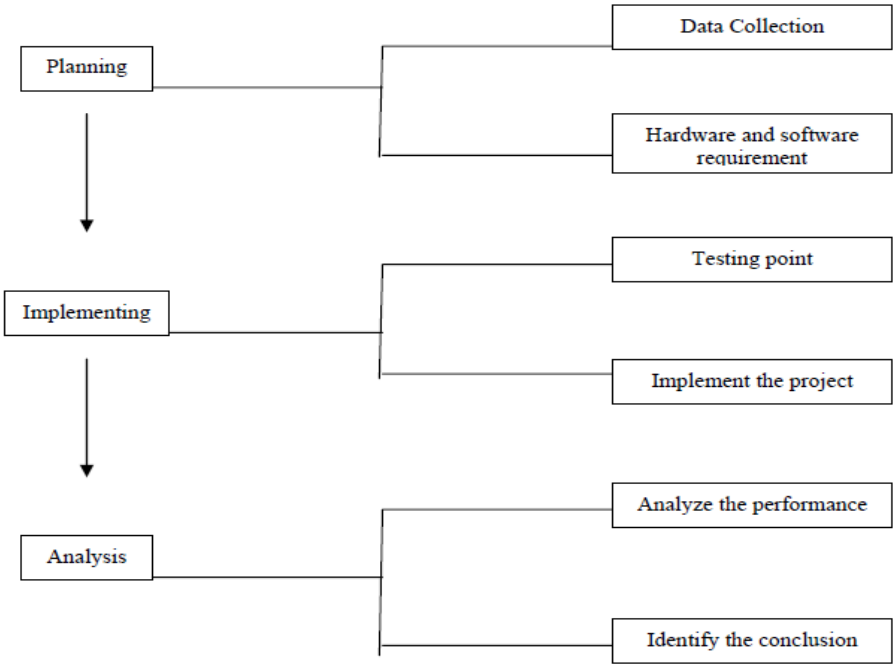


Fig 12: Steps of Methodology

Planning:

To identify all the information and requirement such as hardware and software, planning must be done in the proper manner. The planning phase has two main elements namely data collection and the requirements of hardware and software.

Data collection:

Machine learning needs two things to work, data (lots of it) and models. When acquiring the data, be sure to have enough features (aspect of data that can help for a prediction, like the surface of the house to predict its price) populated to train correctly your learning model. In general, the more data you have the better so make to come with enough rows.

The primary data collected from the online sources remains in the raw form of statements, digits and qualitative terms. The raw data contains error, omissions and inconsistencies. It requires corrections after careful scrutinizing the completed questionnaires. The following steps are

involved in the processing of primary data. A huge volume of raw data collected through field survey needs to be grouped for similar details of individual responses.

Data Preprocessing is a technique that is used to convert the raw data into a clean data set. In other words, whenever the data is gathered from different sources it is collected in raw format which is not feasible for the analysis.

Therefore, certain steps are executed to convert the data into a small clean data set. This technique is performed before the execution of Iterative Analysis. The set of steps is known as Data Preprocessing. It includes -

- Data Cleaning
- Data Integration
- Data Transformation
- Data Reduction

Data Preprocessing is necessary because of the presence of unformatted real-world data. Mostly real-world data is composed of -

- **Inaccurate data (missing data) -** There are many reasons for missing data such as data is not continuously collected, a mistake in data entry, technical problems with biometrics and much more.
- **The presence of noisy data (erroneous data and outliers) -** The reasons for the existence of noisy data could be a technological problem of gadget that gathers data, a human mistake during data entry and much more.
- **Inconsistent data -** The presence of inconsistencies are due to the reasons such that existence of duplication within data, human data entry, containing mistakes in codes or names, i.e., violation of data constraints and much more.

Structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

The Python interpreter and the extensive standard library are freely available in source or binary form for all major platforms from the Python Web site, https://www.python.org/, and may be freely distributed. The same site also contains distributions of and pointers to many free third party Python modules, programs and tools, and additional documentation.

The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications.

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Features:

Python's features include – All these.

- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.

- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − you can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** − Python provides interfaces to all major commercial databases.
- **GUI Programming** − Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** − Python provides a better structure and support for large programs than shell scripting.

Why python emerging as a leader:

There's battle out there happening in the minds of aspiring data scientists to choose the best data science tool. Though there are quite a number of data science tools that provide the much-needed option, the close combat narrows down between two popular languages – Python and R.

Between the two, Python is emerging as the popular language used more in data science applications.

Take the case of the tech giant Google that has created the deep learning framework called tensor flow – Python is the primary language used for creating this framework. Its footprint has continued to increase in the environment promoted by Netflix. Production engineers at Face book and Khan Academy have for long been using it as a prominent language in their environment.

Python has other advantages that speed up it's upward swing to the top of data science tools. It integrates well with the most cloud as well as platform-as-a-service providers. In supporting

multiprocessing for parallel computing, it brings the distinct advantage of ensuring large-scale performance in data science and machine learning. Python can also be extended with modules written in C/C++.

Where Python becomes the perfect-fit:

There are tailor-made situations where it is the best data science tool for the job. It is perfect when data analysis tasks involve integration with web apps or when there is a need to incorporate statistical code into the production database. The full-fledged programming nature of Python makes it a perfect fit for implementing algorithms.

Its packages rooted for specific data science jobs. Packages like Numpy, SciPy, and pandas produce good results for data analysis jobs. While there is a need for graphics, Python's Matplotlib emerges as a good package, and for machine learning tasks, scikit-learn becomes the ideal alternate.

Why is Python preferred over other data science tools?

It is 'Pythonic' when the code is written in a fluent and natural style. Apart from that, it is also known for other features that have captured the imaginations of data science community.

Easy to learn:

The most alluring factor of Python is that anyone aspiring to learn this language can learn it easily and quickly. When compared to other data science languages like R, Python promotes a shorter learning curve and scores over others by promoting an easy-to-understand syntax.

Scalability:

When compared to other languages like R, Python has established a lead by emerging as a scalable language, and it is faster than other languages like Matlab and Stata. Python's scalability lies in the flexibility that it gives to solve problems, as in the case of YouTube that migrated to Python. Python has come good for different usages in different industries and for rapid development of applications of all kinds.

Choice of data science libraries:

The significant factor giving the push for Python is the variety of data science/data analytics libraries made available for the aspirants. Pandas, Stats Models, NumPy, SciPy, and Scikit-Learn, are some of the libraries well known in the data science community. Python does not stop with that as libraries have been growing over time. What you thought was a constraint a year ago would be addressed well by Python with a robust solution addressing problems of specific nature.

Python community:

One of the reasons for the phenomenal rise of Python is attributed to its ecosystem. As Python extends its reach to the data science community, more and more volunteers are creating data science libraries. This, in turn, has led the way for creating the most modern tools and processing in Python.

The widespread and involved community promotes easy access for aspirants who want to find solutions to their coding problems. Whatever queries you need, it is a click or a Google search away. Enthusiasts can also find access to professionals on Code mentor and Stack Overflow to find the right answers for their queries.

Jupyter Notebook (IDE)

The Jupyter Notebook is an **interactive computing environment** that enables users to author notebook documents that include: - Live code - Interactive widgets - Plots - Narrative text - Equations - Images – Video.

These documents provide a complete and self-contained record of a computation that can be converted to various formats and shared with others using email, Dropbox, version control systems (like git/GitHub) or nbviewer.jupyter.org.

Components

The Jupyter Notebook combines three components:

- **The notebook web application**: An interactive web application for writing and running code interactively and authoring notebook documents.

- **Kernels**: Separate processes started by the notebook web application that runs users' code in a given language and returns output back to the notebook web application. The kernel also handles things like computations for interactive widgets, tab completion and introspection.

- **Notebook documents**: Self-contained documents that contain a representation of all content visible in the notebook web application, including inputs and outputs of the computations, narrative text, equations, images, and rich media representations of objects. Each notebook document has its own kernel.

Notebook web application

The notebook web application enables users to:

- **Edit code in the browser**, with automatic syntax highlighting, indentation, and tab completion/introspection.

- **Run code from the browser**, with the results of computations attached to the code which generated them.

- See the results of computations with **rich media representations**, such as HTML, LaTeX, PNG, SVG, PDF, etc.

- Create and use **interactive JavaScript widgets**, which bind interactive user interface controls and visualizations to reactive kernel side computations.

- Author **narrative text** using the Markdown markup language.

- Include mathematical equations using **LaTeX syntax in Markdown**, which are rendered in-browser by MathJax.

Kernels

Through Jupyter's kernel and messaging architecture, the Notebook allows code to be run in a range of different programming languages. For each notebook document that a user opens, the web application starts a kernel that runs the code for that notebook. Each kernel is capable of running code in a single programming language. **Notebook documents**

Notebook documents contain the **inputs and outputs** of an interactive session as well as **narrative text** that accompany the code but are not meant for execution. **Rich output** generated by running code, including HTML, images, video, and plots, is embedded in the notebook, which makes it a complete and self-contained record of a computation.

When you run the notebook web application on your computer, notebook documents are just **files on your local filesystem with a ``.ipynb`` extension**. This allows you to use familiar workflows for organizing your notebooks into folders and sharing them with others.

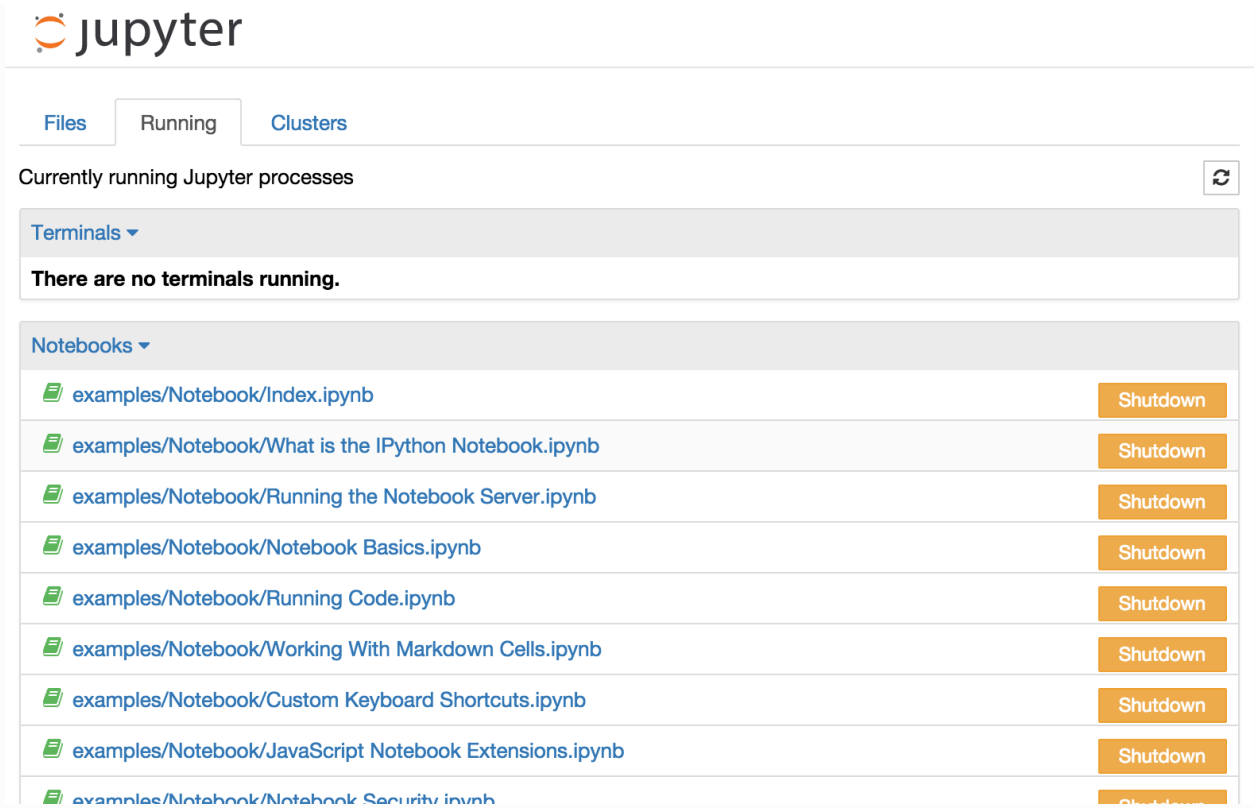
Notebooks consist of a **linear sequence of cells**. There are four basic cell types:

- **Code cells:** Input and output of live code that is run in the kernel
- **Markdown cells:** Narrative text with embedded LaTeX equations
- **Heading cells:** 6 levels of hierarchical organization and formatting
- **Raw cells:** Unformatted text that is included, without modification, when notebooks are converted to different formats using nbconvert.

Internally, notebook documents are **`JSON <https://en.wikipedia.org/wiki/JSON>`__ data** with **binary values `base64 <http://en.wikipedia.org/wiki/Base64>`__** encoded. This allows them to be **read and manipulated programmatically** by any programming language. Because JSON is a text format, notebook documents are version control friendly.

Notebooks can be exported to different static formats including HTML, reStructeredText, LaTeX, PDF, and slide shows (reveal.js) using Jupyter's nbconvert utility.

Furthermore, any notebook document available from a **public URL on or GitHub can be shared** via nbviewer. This service loads the notebook document from the URL and renders it as a static web page. The resulting web page may thus be shared with others **without their needing to install the Jupyter Notebook**.



This view provides a convenient way to track notebooks that you start as you navigate the file system in a long running notebook server.

Chapter 5

Results Analysis

5.1Convolution Neural Network (CNN)

5.1.1CLSTM - Report on Classification

A classification report is in fact a performance review metric in machine learning. It is used to show the accuracy, memory, F1 Measure, and support of the trained classification model. For the sake of clarity, Fig. 13 provides each of the indicators from the CNN classification report.

Metrics	Definition
Precision	Precision is defined as the ratio of positive examples to the sum of such actual and false positives.
Recall	Recall is defined as the ratio of correct positives to all true negatives and false negatives.
F1 Score	A weighted harmonic average of such recall and precision is known as the F1. The projected capacity for the model is higher the closer the F1 score value is near 1.0.
Support	A amount of instances of a class that truly exist in the dataset constitutes the number of supports. It does not differentiate between kinds; it only improves the performance evaluation process.

	precision	recall	f1-score	support
glioma	0.95	0.99	0.97	288
meningioma	0.99	0.94	0.97	265
notumor	0.99	1.00	1.00	291
pituitary	1.00	0.99	0.99	299
accuracy			0.98	1143
macro avg	0.98	0.98	0.98	1143
weighted avg	0.98	0.98	0.98	1143

Fig 13: Classification Report for CNN

The table shows the precision, recall, and F1-score for four different types of brain tumors - glioma, meningioma, notumor, and pituitary. The accuracy of the classification model is reported to be 0.98, indicating that the model correctly classified 98% of the brain tumor cases. The precision for all tumor types is high, ranging from 0.95 to 1.00, indicating that the model made very few false-positive predictions. The recall for all tumor types is also high, ranging from 0.94 to 1.00, indicating that the model identified almost all cases of each tumor type. The F1-score, which takes into account both precision and recall, is also high for all tumor types, ranging from 0.97 to 1.00. Overall, the model seems to be highly accurate and precise in classifying the different types of brain tumors.

5.1.2 CNN- Confusion Matrix

A confusion matrix aids in visualizing the results of a classification task by providing a table arrangement of the different scenarios of the prediction and findings. It given below fig 21. It creates a table with all of a classifier's predicted and actual values.

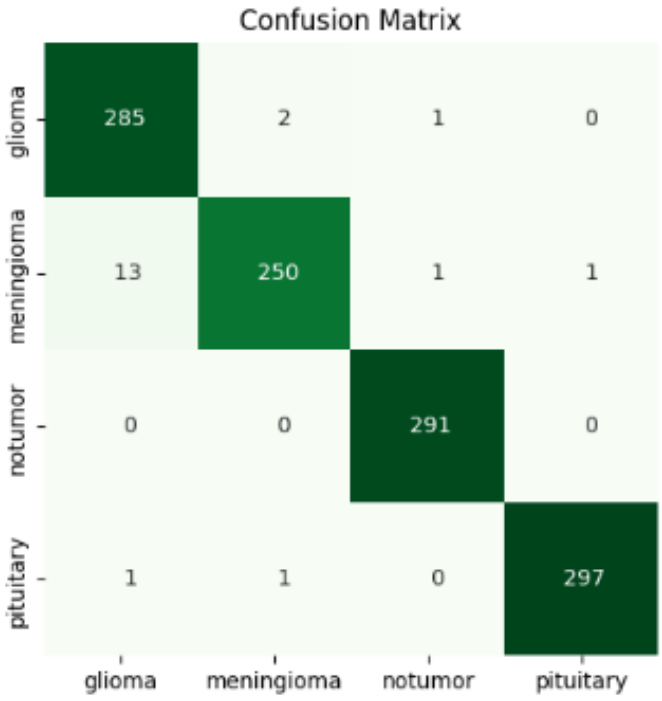


Fig 14: Confusion Matrix for CLSTM

Fig 14 shows the chart for the confusion CNN model. From the confusion matrix understand this model is performing excellent. Four thousand photos are provided in total and this model got 98%

accuracy. 1143 photos are provided in total. And this model correctly predicted 1123 images and only 20 images predicted wrongly, this model got 98% accuracy.

Chapter 6

6.1 Conclusion

Tumor detection and classification using image processing is a powerful tool in the diagnosis and treatment of cancer. This technique has several advantages over traditional methods, including its non-invasive nature and ability to analyze large amounts of data quickly and accurately.

Image processing algorithms can detect and classify tumors in medical images such as MRI, CT, and PET scans, and provide physicians with valuable information about the size, location, and characteristics of the tumor. This information can be used to develop personalized treatment plans and improve patient outcomes.

However, there are some limitations to tumor detection and classification using image processing. These include the need for high-quality images and the potential for false positives or false negatives. Additionally, image processing algorithms may not be able to detect tumors in certain areas of the body or in certain types of cancer.

Overall, tumor detection and classification using image processing is a promising technology that has the potential to revolutionize the way cancer is diagnosed and treated. Ongoing research and development in this area will likely lead to even more accurate and effective tools for detecting and classifying tumors.

References

[1] Komal Sharma, Navneet Kaur, " Comparative Analysis of Various Edge Detection Techniques" , International Journal of Advanced Research inC omputer Science and Software Engineering, IJARCSSE, ISSN: 2277 128X, Volume 3, Issue 12, December 2013.

[2] Suchita Goswami, Lalit Kumar P. Bhaiya, " Brain Tumor Detection Using Unsupervised Learning based Neural Network" , IEEE International Conference on Communication Systems and Network Technologies,2013.

[3] Walaa Hussein Ibrahim, Ahmed Abdel Rhman Ahmed Osman, Yusra Ibrahim Mohamed, "MRI Brain Image Classification Using Neural Networks" ,IEEE International Conference On Computing, Electrical and Electronics Engineering, ICCEEE,2013.

[4] Dipali M. Joshi, N. K. Rana, V. M. Misra, " Classification of Brain Cancer Using Artificial Neural Network" , IEEE International Conference on Electronic Computer Technology ,ICECT ,2010.

[5] A. Prabin and J. Veerappan, “Computer Assisted Diagnosis of Brain Tumor in MRI image using Texture features as Input to AdaBoost Classifier”, Research Journal of Applied Sciences, Engineering and Technology 8(24): 2374-2380, 2014

[6] Sahar Ghanavati, Junning Li, Ting Liu, Paul S. Babyn, Wendy Doda, George Lampropoulos, “Automatic Brain Tumor Detection In Magnetic Resonance Images” IEEE 2012

[7] V. Trinadh babu, Sk. Salma Begum, “An Adaptive MRI Tumor Detection Using NeuralNetwork Based Adaboost Algorithm”, International Journal of Computer Science and Information Technologies, Vol. 6 (1) , 2015

[8] Prachi Gadpayleand, P.S. Mahajani, "Detection and Classification of Brain Tumor in MRI Images ", International Journal of Emerging Trends in Electrical and Electronics, IJETEE – ISSN: 2320-9569, Vol.5, Issue. 1, July-2013.

Appendix A

Appendix A consists of technical specifications required to implement this research.

HARDWARE REQUIREMENTS

- System Processor : Core i3 / i5
- Hard Disk : 500 GB.
- Ram : 4 GB.

✓ *Any desktop / Laptop system with above configuration or higher level.*

SOFTWARE REQUIREMENTS

- **Operating system** : Windows 8 / 7/10
- **Coding Language** : Python
- **Tools** : Anaconda
- **IDE** : Jupyter Notebook

- **Programming Languages**
  - Python Programming